Don’t Stop Pretraining-Adapt Language Models to Domains and Tasks 带读

Don’t Stop Pretraining: Adapt Language Models to Domains and Tasks：继续预训练（continued pretraining）把“领域差异”变成可控增益

0. TL;DR（≤120字）

- 贡献一句话：提出并系统验证 DAPT/TAPT：继续预训练（continued pretraining）可稳定提升跨领域下游任务表现。
- 适用一句话：当你的下游数据来自“预训练没覆盖/覆盖不足”的新领域（domain）或强任务偏置分布（task distribution）时。
- 风险一句话：领域不匹配或过度贴合任务分布，可能负迁移（negative transfer）或灾难性遗忘（catastrophic forgetting）。

---

1. 背景与动机：他们到底在“卡什么”

任务背景：用 5~10 句讲清语境

这篇论文研究一个很“工程现实”的问题：你拿到一个强大的通用预训练语言模型（pretrained language model, PLM），但你的任务数据来自一个风格、词汇、格式都很不一样的领域（domain），比如生物医学、计算机论文、新闻、商品评论。
通用预训练并不等于“所有领域都学会”：不同领域的术语（term）、缩写（abbreviation）、句法偏好（syntax preference）、标点/格式习惯（formatting conventions）差异很大。
而下游标注数据往往不多，尤其在低资源（low-resource）设置下，直接 fine-tuning 很像“用少量标注硬改语言常识”，不稳定、易过拟合、还可能学不到关键术语。
作者把“数据分布关系”拆成三层：预训练分布（pretraining distribution）→目标领域分布（target domain distribution）→任务分布（task distribution）。
关键点在于：任务数据通常是从目标领域分布里“非随机采样”来的，因此**任务分布**与**领域分布**并不等价。
所以继续预训练到底应该对齐“领域”，还是对齐“任务”？两者能否叠加？这篇论文给出系统实验回答。

痛点拆解：为什么已有方法不够

1. 只 fine-tuning 不够稳：少量标注难以补齐领域语言统计（domain-specific language statistics）。
2. 既有“领域预训练”工作不够系统：常在单领域或较弱基础模型上验证，难外推到强通用 PLM + 多领域的现实。
3. 不知道该在什么粒度上继续预训练：用大量领域无标注数据（DAPT）还是用任务训练集文本（TAPT）？

作者的关键洞见：一句话点题

不要停在通用预训练：继续预训练（continued pretraining）在“领域层面（DAPT）+任务层面（TAPT）”都可能带来可叠加收益。

---

2. 问题定义与符号（足够让人复现）

输入/输出：清晰列出

论文主体实验围绕**文本分类（classification）**的跨领域设置展开（多领域、多任务）。

- 输入：token 序列 $x = (x_1, \ldots, x_n)$

- 输出：类别标签 $y \in \{1, \ldots, K\}$

继续预训练使用**掩码语言建模（Masked Language Modeling, MLM）**：

- 输入：无标注文本 x
- 输出：被 mask 的 token 的条件分布

目标函数/评价指标：必要时给公式

继续预训练（MLM）目标：

$$\mathcal{L}_{\mathrm{MLM}}(\theta) = -\sum_{i \in M} \log p_\theta(x_i \mid x_{\setminus M})$$

下游分类 fine-tuning 目标：

$$\mathcal{L}_{\mathrm{cls}}(\phi) = -\log p_\phi(y \mid x)$$

评价指标随任务而变（如 Acc / F1 等）；关键在于**同一任务下**比较不同继续预训练策略的增益。

关键假设：作者默认了什么

1. 你已经有一个强通用 PLM（如 RoBERTa 一类）。
2. 你能获得一定量目标领域无标注语料（unlabeled in-domain corpus），或至少能使用任务训练集文本本身（忽略标签，当作无标注）。
3. 继续预训练与下游 fine-tuning 是串联两阶段（先自监督，再监督微调）。

---

3. 方法总览：一张“脑内结构图”

论文把“继续预训练”拆成两个可组合插槽：

```
（0）通用预训练LM
        |
        | 可选：DAPT（domain-adaptive pretraining）领域继续预训练
        v
（1）领域适配后的LM
        |
        | 可选：TAPT（task-adaptive pretraining）任务继续预训练
        v
（2）任务适配后的LM
        |
        | 下游监督微调 fine-tuning
        v
（3）任务模型 + 评测
```

每一步的作用与替代方案：

- DAPT：让模型学会“这个领域怎么说话”（术语、格式、风格、搭配）。替代：换用领域专用 LM（若存在且匹配）。
- TAPT：让模型贴近“这道题的数据分布怎么写文本”（任务采样偏差、模板化表达）。替代：强正则 fine-tuning、数据增强等，但不一定补“语言统计”。
- 数据选择（可选增强）：当你有很多领域数据但算力不够或数据太杂，用检索/筛选找到更像任务的子集再 TAPT。

4. 核心方法逐段讲解（按论文逻辑走）

4.1 模块 A：DAPT（domain-adaptive pretraining）

它做什么

在目标领域无标注语料上继续做 MLM 训练，让模型参数从“通用语言统计”偏移到“领域语言统计”。

直觉（为什么有效）

领域差异体现在：

- 高频术语、缩写、固定搭配
- 句法偏好：长句、被动、列表结构等
- 标点/格式：括号、引文、编号、模板化段落
  DAPT 用自监督把这些统计灌回模型，让下游监督学习更像在学“任务边界”，而不是补语言常识。

形式化（伪代码）

```
# DAPT: continue pretraining on in-domain corpus input: pretrained LM parameters θ0, domain corpus D_domain (unlabeled) θ ← θ0 for step in 1..S_dapt: x ~ D_domain x_masked, M ← mask(x) loss ← - Σ_{i∈M} log p_θ(x_i | x_masked) θ ← AdamUpdate(θ, loss) return θ
```

工程细节（实现坑、默认超参、复杂度）

- 成本：DAPT 通常需要更大语料与更长训练步数；对存储与吞吐要求更高。
- 关键旋钮：训练步数/epoch 与学习率（lr）最敏感。
- 坑 1：领域语料“同名不同域”（子域差异大）会稀释收益甚至转负。
- 坑 2：训练过久/lr 过大可能导致灾难性遗忘，损害通用能力并影响下游稳定性。

4.2 模块 B：TAPT（task-adaptive pretraining）

它做什么

用任务训练集文本本身（忽略标签）做 MLM 继续预训练，再 fine-tune。

直觉（为什么有效）

任务数据分布往往比领域分布更“偏”：

- 采样偏差：论坛帖、摘要体、模板化标题
- 标注机制带来的语言偏好：筛选规则、标注员习惯
  TAPT 是对任务分布的“对口校准（calibration）”，让监督微调更专注于任务决策。

形式化（伪代码）

```
# TAPT: continue pretraining on task training texts (ignore labels) input: LM parameters θ_in, task training texts D_task_text θ ← θ_in repeat for some epochs or steps: x ~ D_task_text do MLM update as in DAPT return θ
```

工程细节

- 成本极低：通常比 DAPT 小一个量级，非常适合算力受限（如单卡 8GB）。
- 坑：任务训练集很小且模式强，TAPT 训练太久可能过拟合“表面格式”，导致泛化下降。

- 依然最敏感旋钮：训练步数/epoch + lr；建议配合 dev 监控与早停（early stopping）。

4.3 模块 C：DAPT + TAPT 的组合（可叠加性）

它做什么

先 DAPT 把模型搬到领域附近，再 TAPT 推到任务分布上，最后 fine-tuning。

直觉

- DAPT：解决“领域语言”
- TAPT：解决“任务采样与表达方式”
  二者针对不同层级错配（distribution mismatch），往往互补。

工程要点

- 算力允许：DAPT→TAPT 通常是更强主路线。
- 算力不允许：先 TAPT 也能吃到明显收益，是优先级最高的低成本改造。

4.4 模块 D：数据选择（kNN-TAPT / Curated-TAPT）

当你有大量领域无标注语料，但：

- 算力不够跑完整 DAPT；或
- 语料太杂，直接 DAPT 会把噪声学进去；
  就需要“选更像任务的子集”再继续预训练。

4.4.1 Curated-TAPT（人工挑选）

手工挑选更贴近任务风格/体裁的无标注文本子集进行继续预训练。

4.4.2 kNN-TAPT（自动近邻选择）

用文本嵌入（embedding）把任务样本映射到向量空间，从大语料中检索最近邻（kNN），形成更像任务的子集再 TAPT。

```
# kNN-TAPT (conceptual) input: task texts T, large corpus C, embedding f(·), neighbors k E_T = { f(t) for t in T } for each t in T: N_t = kNN_search(f(t), {f(c) for c in C}, k) S = union of N_t continue_pretrain_on(S) # MLM continued pretraining fine_tune_on_labeled_task()
```

工程要点：

- 子集大小要控：太小易过拟合，太大退化成“杂乱 DAPT”。
- 表示函数要稳：嵌入质量差会引入检索噪声，伤害后续预训练。

---

5. 训练/推理细节（工程读者最关心的部分）

数据处理：tokenizer/归一化/噪声处理等

- 继续预训练通常沿用基础模型的 tokenizer 与掩码策略（masking scheme），先别改太多。
- 真正决定收益的是：你提供的无标注语料是否“代表目标分布”，以及训练步数/学习率是否合理。

训练策略：batching, lr, schedule, 正则, dropout...

- 工程建议：先固定一套默认配方，仅扫 `lr × steps/epochs` 小网格，找最稳的增益区间。
- 小数据 TAPT 更敏感：更需要 dev 监控与早停，避免“学会格式，丢了泛化”。

推理策略：beam, length penalty, constraints...

论文主实验是分类，不涉及 beam search；迁移到生成任务时需要自行补齐（见第 8 节）。

复杂度与资源：显存/时间/吞吐

经验性结论（工程导向）：

- TAPT 是最便宜、最先做的继续预训练插槽。
- DAPT 更贵，但在领域差异大时往往更强；可先做 “DAPT-lite” 验证增益，再决定是否扩大。

---

6. 实验设计：他们怎么证明“不是玄学”

数据集与划分：泄漏风险、split 关注点

核心逻辑：固定下游任务与划分，仅改变继续预训练策略（DAPT/TAPT/组合/数据选择），从而隔离变量，减少“玄学解释空间”。

Baseline：有哪些对照

- 直接 fine-tune（无继续预训练）
- DAPT 后 fine-tune
- TAPT 后 fine-tune
- DAPT + TAPT 后 fine-tune
  - （可选）数据选择变体：Curated/kNN-TAPT

消融：最关键结论

1. DAPT 与目标域距离越大时越关键。
2. TAPT 往往可在 DAPT 之上再提升，说明任务分布与领域分布不同。
3. 数据选择在算力/语料受限时提供有效折中。

结果解读：解释趋势

一句话工程解释：继续预训练的收益来自降低分布错配（distribution mismatch）；DAPT 对齐领域语言统计，TAPT 对齐任务采样与表达分布。

失败案例：推断并标注

- 【推断】领域语料与任务子域不匹配 → DAPT 稀释/负迁移
- 【推断】TAPT 训练太久 → 过拟合任务表面模式
- 【推断】lr/步数不当 → 继续预训练不稳定，表示崩坏或无效更新

---

7. 批判性复盘：这篇论文哪些地方可能不靠谱

强依赖的前提

- 你拿到的无标注语料必须“真同域”；否则继续预训练可能学到无关统计。

可能的负迁移点

- 任务分布极窄：TAPT 可能把模型推向“训练集写作习惯”，遇到测试分布轻微变化就掉分。
- 领域语料太杂：DAPT 可能吸收噪声，收益变小甚至转负。
- 训练过久：灾难性遗忘（catastrophic forgetting）风险上升。

复现实验最容易踩的坑

- 最敏感旋钮仍是 `steps/epochs` 与 `lr`；建议小网格扫参 + dev 监控 + 早停。
- 小数据 TAPT 不要贪多：你在“校准分布”，不是“再造预训练”。

与相邻方法的边界（什么时候别用它）

- 当目标任务已接近原预训练分布：DAPT 边际收益可能很小，算力更该投向更强下游建模或数据清洗。
- 当拿不到任何靠谱的同域无标注文本：DAPT 不成立；更适合做数据增强、参数高效微调（PEFT）或换基础模型。

---

8. 迁移到 Deep Past（Akkadian→English）的落地方案

> 迁移的核心不是照搬 RoBERTa+MLM，而是照搬“插槽思想”：在**领域层级**与**任务层级**做低成本分布对齐，再做监督训练。

我们的 pipeline 插入点：tokenizer / model / training / decoding / postprocess / eval

- Tokenizer：Akkadian 转写（transliteration）会包含符号、连字符、数字、缩写；tokenization 很可能决定上限。
- Model：更自然用 encoder-decoder（seq2seq）模型（T5/mBART/ByT5 等）；继续预训练目标用其去噪/自监督目标（denoising objective）。
- Training 插槽：
  - TAPT：直接用比赛训练集的 source/target 文本做短程继续预训练（优先级最高，成本最低）。
  - DAPT：若能获得合规“更大规模”转写无标注语料或相关领域文本，可做 DAPT-lite 再叠加 TAPT。
- Decoding：翻译任务需要 beam search、length penalty、（可选）术语/格式约束；论文不覆盖，这里要单独工程化。
- Eval：BLEU / chrF++（以及比赛官方最终指标） + 诊断指标（长度分布、符号合法率、复制率等）。

最小可复现实验（MVP）

对照组设置

1. Baseline：直接 fine-tune 一个小型 seq2seq（适配 8GB）。
2. +TAPT：训练集文本上做短程继续预训练 → fine-tune。
3. +DAPT-lite（可选）：若有合规外部无标注转写语料：DAPT-lite → TAPT → fine-tune。

需要改动的代码/配置点（按模块列）

- 数据管线：新增 `continue_pretrain_dataset`（读无标注序列）
- 训练脚本：新增 `stage={dapt,tapt,finetune}` 三段式流程
- 记录与复现：固定 seed、版本号、数据清洗规则；每段训练都产出可回放日志

验收指标（BLEU/chrF++/几何均值 + 额外诊断指标）

- 主指标：BLEU / chrF++（按比赛要求选择主提交指标）
- 诊断：
  - 输出长度分布 vs 参考长度分布
  - 特殊符号合法率（括号/连字符/数字格式等）
  - 复制率（source-copy ratio）：输出中直接复制转写片段的比例（常见退化）

预计算力（8GB 场景）与降级方案

- 先做 TAPT：最便宜、最可能立刻见效。
- DAPT 做 lite：少步数 + 混合精度 + 梯度累积；把它当“词表/符号习惯校准”，别追求大规模。
- 仍吃紧：使用参数高效微调（PEFT，例如 LoRA）作为工程补丁。

进一步迭代路线（最多 5 条，按优先级）

1. 先把 TAPT 跑出稳定增益：扫 `lr × steps` 找甜点区。
2. 上 DAPT-lite 并监控负迁移：dev 曲线 + 早停。
3. 数据选择：若外部语料很杂，用检索/筛选做“更像任务”的子集再继续预训练。
4. decoding 约束工程化：格式/符号一致性约束，减少退化输出。
5. 分桶错误分析：按体裁/时期/文本长度等 bucket 定位“谁受益谁受伤”。

---

9. 读完该做什么（行动清单）

立刻能做的 3 件事（1 小时内）

1. 把训练流程拆成三段：`tapt_pretrain → finetune → evaluate`，先不碰 DAPT。
2. 明确 TAPT 数据输入：只用训练集文本；固定清洗规则（空格、连字符、特殊符号）。
3. 固定日志模板：每段都输出 step/lr/loss、dev 指标、输出长度统计与复制率。

需要半天的 3 件事

1. TAPT 小扫参：`lr × steps/epochs` 网格（少量点位即可）。
2. tokenizer 对照：SentencePiece vs byte-level（至少两组）。
3. 错误分析脚本：符号合法率/复制率/长度偏差报表化（否则无法解释涨分来源）。

需要 1~3 天验证的 3 件事

1. 若有合规外部语料：DAPT-lite，并验证是否出现负迁移/遗忘。
2. 尝试“kNN 选数据再继续预训练”的路线（数据规模变大后更有价值）。
3. 工程化实验矩阵：`{tokenizer}×{tapt_steps}×{dapt_lite?}×{decode_params}`，保证可复现与可回滚。